GREENROOM · PRICING ENGINE REFERENCE · MAY 2026

Smart Guaranteed Pricing vs Flat Deals

A flat deal is a contract shape — a fixed dollar guarantee with no percentage upside. Smart Guaranteed Pricing (SGP) is a pricing engine — the seven-step model that decides what the dollar number on a flat contract should be when you're converting a vs / % of net / % of gross / door deal into one. This document explains the distinction, then walks the engine end to end, exactly as it runs in `artifacts/api-server/src/lib/smartGuarantee.ts`.

Sourced from `lib/smartGuarantee.ts`, `lib/smartSwitch.ts`, and `lib/switchSavings.ts`. All step numbers, source-enum names, and tier thresholds are taken from the live code.

1. The distinction, in one paragraph

A **flat deal** is one of the five contract shapes the venue can sign — alongside `vs`, `percentage_of_net`, `percentage_of_gross`, and `door`. It says "the artist gets $X, no settlement-night arithmetic." Nothing about a flat deal explains where the X came from; historically it came from Mariana's gut, the agent's ask, and a 5-minute back-of-envelope.

Smart Guaranteed Pricing is what produces the X. It reads the deal's structure (deal type, guarantee, percentage, expense cap), looks up comparable history at this venue, runs a deterministic seven-step waterfall, and emits a single suggested flat dollar amount — together with a confidence tier, an insurance tier, a basis sentence, and a full audit JSON. The booker can then either accept it (the deal converts to a clean flat in the DB) or keep negotiating with the agent against a number that has receipts.

> **Restated:** Flat is the destination. SGP is the map. A flat deal entered by hand and a flat deal produced by SGP look identical in the database, but only the SGP one carries a paper trail.

2. Why SGP exists at all

- **vs / %-of-net deals win the percentage clause most nights.** Across this venue's 137 historical vs/$1–5K settlements, the percentage clause out-paid the guarantee 87.6% of the time, with mean overshoot ~$2,338/show. A flat-equivalent that ignored that fact would systematically underpay.

- **Door deals lose money for the venue 93% of the time** at The Crescent. Converting them needs a structurally different shape (a floor + capped split, not a flat) — SGP and Smart Switch coordinate that.
- **Bookers were doing this math anyway,** in a spreadsheet, inconsistently, with no audit. SGP standardizes the calculation and surfaces the inputs so the agent's pushback has somewhere to land.

3. The seven-step waterfall

Every call to `generateGuarantee(showId)` runs these seven steps in order. The intermediate values are persisted on the `guarantee_suggestions` row and exposed via the `auditJson` field, so every number on screen can be traced back to its inputs.

1

EXPECTED GROSS

How much money will the door bring in? Picked from a six-level source waterfall (first hit wins):

SOURCE	PICKED WHEN
`artist_at_venue`	≥1 prior show by this artist at this venue (uses their mean gross here)
`artist_anywhere`	≥1 prior show by this artist (any venue)
`agent_history`	≥3 prior shows by this agent at this venue
`cell_mean`	Mean gross across all past dealType × sizeBucket cells matching this deal
`venue_mean`	Mean gross across all past shows at the venue
`capacity_proxy`	Fallback synthesis: `venueCapacity × $30 × 0.6 load`

2

TICKETING FEES

```
ticketingFees = expectedGross × 0.10
```

Flat 10% ticketing fee. This is a venue-level constant; if your platform changes, edit `TICKETING_FEE_RATE`.

3

NET AFTER FEES

```
netAfterFees = expectedGross - ticketingFees
```

4

CAPPED EXPENSE ESTIMATE

Like step 1, the raw expense number comes from a source waterfall:

SOURCE	PICKED WHEN

`artist_history_2plus`	≥2 prior shows by this artist (mean of their actual expense totals)
`artist_history_1`	Exactly 1 prior show by this artist
`agent_history`	≥1 prior show by this agent
`genre_p75`	P75 of expenses across the artist's genre
`venue_mean`	Mean of all past expenses at the venue

Then the raw value is **capped** twice — first by the deal's own `expenseCap` if signed, then by a per-bucket default (`$0-1K → $800`, `$1-5K → $1,500`, `$5-15K → $3,500`, `$15K+ → $7,500`). The lower of the two is the `effectiveCap`:

```
expenseEstimate = min(raw, effectiveCap)
effectiveCap   = min(deal.expenseCap ?? ∞, defaultCap[bucket])
```

5

NET BASE

```
netBase = max(0, netAfterFees - expenseEstimate)
```

This is the pool the percentage clause would draw from for vs / % of net / door deals.

6

PROJECTED PERCENTAGE PAYOUT

What the artist would walk away with under the structured part of the deal, given the projection:

```
if dealType == "percentage_of_gross":  basis = expectedGross
elif dealType == "door":               basis = netBase   (pct defaults to
0.70 if unset)
else (vs, "% of net"):                 basis = netBase

percentagePayout = max(0, pct × basis)
```

A defensive normalization divides `pct` by 100 if a legacy row stored "85" instead of "0.85", so the math doesn't blow up.

7

WINNER & SUGGESTED PRICE

```
winnerValue   = max(guarantee, percentagePayout)
suggestedPrice = roundTo50(winnerValue)
breakevenGross = (suggestedPrice + expenseEstimate) / (1 - 0.10)
```

SGP picks whichever side of the deal the artist would have won that night and proposes that as the flat. The `winnerMargin` (absolute gap between the two) feeds the tier calculation in the next section.

4. Confidence & insurance tiers

Confidence tier (how much we trust the number)

Computed by `computeConfidenceTier(artistShowCount, agentShowCount, winnerMargin, hasGenreData)`:

TIER	RULE	MEANING
A	≥3 prior shows by this artist and winnerMargin > $200	"We've seen this artist enough that the projection is solid, and the winning side wasn't a coin flip."
B	≥1 prior show by this artist or ≥3 by this agent	"Some signal, but not enough to put it on the front page without a band."
C	≥1 prior show by this agent or genre data exists	"Indirect signal only. Use as a starting point, not a number."
D	None of the above	"First-time artist, first-time agent, no genre history. Treat as a guess."

Insurance tier (how risky the show is for the venue)

A separate, orthogonal dimension. `computeInsuranceTier()` classifies the venue's downside risk:

TIER	RULE
1	Non-door deal, confidence A, and ≥$1,500 cushion between projected net (after fees + expense) and the suggested price
2	Cushion ≥$500, confidence A or B, non-door
3	Cushion <$500 (near-breakeven), or confidence C
4	Confidence D, or any door deal (door is always tier 4 — see §2)

```
cushion = expectedGross × (1 - 0.10) - expenseEstimate - suggestedPrice
```

5. What SGP emits

A single call to `generateGuarantee()` returns a row that is persisted on `guarantee_suggestions` and includes:

FIELD	MEANING
`agentGuarantee`	The dollar guarantee in the signed contract
`suggestedPrice`	The single number SGP proposes (rounded to nearest $50)

`delta`	`suggestedPrice - agentGuarantee`
`expectedGross`, `expectedGrossSource`	Step 1 output + which source the waterfall picked
`ticketingFees`, `netAfterFees`	Steps 2 & 3
`expenseEstimate`, `expenseSource`, `expenseCap`	Step 4 — including the effective cap that was applied
`netBase`, `percentagePayout`	Steps 5 & 6
`winner`, `winnerMargin`, `breakevenGross`	Step 7
`artistShowCount`, `agentShowCount`	Inputs to the tier formula
`confidenceTier`, `insuranceTier`	§4
`basis`	One-sentence English summary the UI can paste into a counter-offer email
`auditJson`	The full step-by-step record (inputs + step1..step7) — every number on screen is reachable from here

6. Worked example

Take a hypothetical vs deal at the $1–5K bucket: `guarantee = $4,000`, `pct = 0.85`, signed expense cap = $2,000. The artist has 4 prior shows at the venue.

```
Step 1  expectedGross   = $9,000   (source: artist_at_venue, n=4)
Step 2  ticketingFees   = $900     (= 9,000 × 0.10)
Step 3  netAfterFees    = $8,100
Step 4  raw expense     = $1,700   (source: artist_history_2plus)
        defaultCap[$1-5K] = $1,500
        effectiveCap    = min($2,000, $1,500) = $1,500
        expenseEstimate = min($1,700, $1,500) = $1,500
Step 5  netBase         = $8,100 - $1,500 = $6,600
Step 6  percentagePayout = 0.85 × $6,600 = $5,610
Step 7  winnerValue     = max($4,000, $5,610) = $5,610   (winner: percentage)
        suggestedPrice  = roundTo50($5,610) = $5,600
        breakevenGross  = ($5,600 + $1,500) / 0.90 ≈ $7,889
Tiers   confidenceTier  = A   (4 prior shows ≥ 3, winnerMargin $1,610 > $200)
        cushion         = 9,000 × 0.90 - 1,500 - 5,600 = $1,000
        insuranceTier   = 2   (non-door, tier A, cushion < $1,500)
```

The booker sees: "Smart Guaranteed Price **$5,600**, tier A · insurance 2. Projected $5,610 % payout exceeds the $4,000 guarantee by $1,610, so SGP picks the percentage side rounded to $50." If the agent pushes back, every input is one click away in the audit JSON.

7. Where SGP plugs into the rest of the app

Smart Switch ($1–5K vs / % of net)

`lib/smartSwitch.ts` routes through `generateGuarantee()` for the `vs` and `% of net` deals in the `$1–5K` bucket. If SGP returns tier **A** or **B**, the suggested flat is the SGP price (source: `sgp_engine`). If SGP returns tier C/D, Smart Switch falls back to anchoring on the contract guarantee (source: `guarantee_amount`) rather than emitting a low-confidence number.

Switch Savings backtest (Insights)

`lib/switchSavings.ts` re-runs SGP against every past settled `vs / pn / door` deal in the last N months using only data available before the show, then compares the counterfactual SGP-driven payout against what actually settled. That's the "Money saved / Time saved" panel on the Insights tab.

Show detail panel

On any upcoming `vs / pn / % gross / door` show whose settlement is in an actionable stage, the right rail renders the SGP card directly: suggested price, tier chip, insurance chip, basis sentence, and a "show the math" disclosure that pretty-prints the audit JSON.

Insurance pricing brief

The May 2026 insurance pricing brief (`exports/greenroom-insurance-pricing.pdf`) prices Product 2 directly off SGP backtest output — the `SGP - actual` gaps across the full 12-month corpus (n=156, median gap $870, P75 $1,569). Without SGP there is no Product 2.

8. Side-by-side

	FLAT DEAL (ENTERED BY HAND)	FLAT DEAL PRODUCED BY SGP
Contract shape in DB	`dealType = "flat"`, `guaranteeAmount = X`	Identical: `dealType = "flat"`, `guaranteeAmount = X`
Where X came from	Booker's head, agent's ask	Seven-step waterfall over comparable history
Auditability	None	Full `auditJson` blob — every input, every source, every step
Confidence label	—	A / B / C / D with explicit rule
Range disclosure	—	Smart Switch surfaces P10–P90 band; demotes display tier when the band > $1,000

Risk label	—	Insurance tier 1–4 with cushion math
Floor protection	—	Smart Switch enforces `suggestedFlat ≥ contractGuarantee` — converting never underpays the artist vs the signed number
Counter-offer copy	—	`basis` sentence is generated and pastable
Backtestable	No	Yes — `switchSavings.ts` replays the engine against the entire 12-month corpus

> **One thing flat deals do that SGP does not:** when an artist is genuinely new to the room — first-time artist, first-time agent, no genre history — SGP returns confidence **D** and Smart Switch refuses to emit a single suggested flat. In that case the booker is back to a hand-entered flat, and that's the correct behavior. The engine knows when it doesn't know.

9. The one-line takeaway

A flat deal is a contract type with no math. Smart Guaranteed Pricing is the math that decides whether the flat number on a converted contract is honest — a seven-step deterministic waterfall over comparable venue history, surfaced with a confidence tier, an insurance tier, a P10–P90 band, and a full audit JSON. Both end up as `dealType="flat"` in the DB. Only one of them has receipts.

References: `artifacts/api-server/src/lib/smartGuarantee.ts` (generateGuarantee, computeConfidenceTier, computeInsuranceTier, resolveExpectedGross, resolveExpense), `artifacts/api-server/src/lib/smartSwitch.ts` (computeTier, generateSuggestion, switchAppliesTo), `artifacts/api-server/src/lib/switchSavings.ts` (backtest). Regenerate this PDF with: `python3 -c "from weasyprint import HTML; HTML('exports/greenroom-sgp-vs-flat.html').write_pdf('exports/greenroom-sgp-vs-flat.pdf')"`.